Photonic Interconnects for TPU-Scale LLM Accelerators

AccelForge modeling of Lightmatter-style topologies, compiler-visible dataflows, and 9216-chip Gemini-class training/inference

Pranav Sitaraman

6.5930 Hardware Architecture for Deep Learning

Final Project Presentation

Project 12: Evaluate a New Technology

Emerging technology: Lightmatter Passage photonic interconnects

One-Slide Story

72.21%

Best 16-node energy reduction

EVK100 optical crossbar vs electrical 4x4 mesh on shared-DRAM multicast.

5 / 16

Changed distributed mappings

Photonic links make shared-DRAM multicast optimal for prefill-style traffic.

14.50×

Best SuperPod inference speedup

Gemini 3 Deep Think proxy on 9216 Ironwood chips with M1000 full stack.

Thesis

Photonic interconnects are not just cheaper wires. High bandwidth, flatter topology, and native multicast can change the memory placement and compiler dataflow that should be chosen for distributed inference. Training still improves, but becomes compute dominated once collective traffic is reduced.

All headline numbers come from the checked-in pipeline: `make validate`, `make study`, `make inference`, `make training`, and `make superpod`.

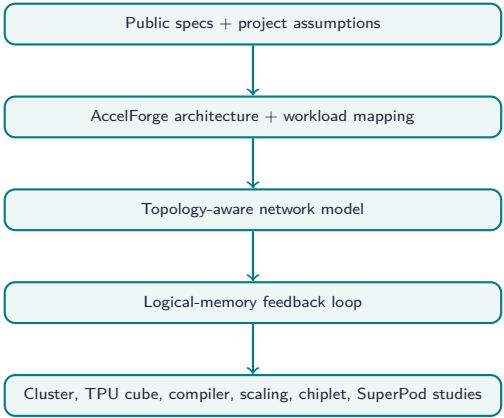
Narrative Arc

The class question

- AccelForge tells us how a workload maps to a memory hierarchy.
- The project asks what happens when the logical memory level is actually a cluster of accelerators joined by a real network.
- The network is fed back into the logical memory cost, then the mapper is rerun until stable.

The research question

- Does replacing electrical ICI with photonics only scale energy and latency?
- Or does it make new distributed mappings and compiler choices optimal?



What Was Modeled and Where It Came From

Photonic technology points

Tech	BW	Energy	MC
Electrical 800G	0.8T	10.0	no
Chiplet 3.5T	3.5T	6.0/hop	no
EVK50	0.8T	4.6	yes
EVK100	3.2T	1.9	yes
L200	16.0T	4.5	yes
L200X	32.0T	4.8	yes
M1000	57.3T	2.3	yes+OCS

MC = native multicast. BW is one-way per modeled link/chip.

Source trail

- Lightmatter EVK50, EVK100, L200/L200X, M1000 product and SC25/OCP material.
- Google TPU v5p and TPU7x/Ironwood documentation plus Hot Chips 2025 rack tutorial.
- XLA/GSPMD motivation from PyTorch/XLA SPMD documentation.
- Gemini 3/3.1/Flash/Deep Think public posts: used only to motivate proxy workloads, since parameter counts are not public.
- Advanced chiplet baseline: prompt-specified 3.5 Tb/s one-way, 6 pJ/bit per hop.

Source: `results/sources.csv`; `results/summary.md`

Codebase and Reproduction Surface

Main implementation

File	Role
`scripts/lightmatter_interconnects.py`	Topologies, technology catalog, AccelForge loop, compiler model, scaling, chiplet, SuperPod studies, CLI.
`scripts/generate_lightmatter_notebook.py`	Rebuilds the executed exploratory notebook.
`arch/tpu_lightmatter_cluster.yaml`	Logical cluster memory, node array, HBM, UB, register, MAC.
`workload/single_matmul.yaml`	Parameterized GEMM template for M/K/N/bits.
`tests/test_lightmatter_project.py`	70 unit/integration

Exact commands

Dockerized workflow

```
make validate
make study
make inference
make training
make superpod
make notebook
```

All targets run inside `timeloopaccelergy/accelforge:latest-amd64`. The notebook has 80 executed cells and the current test suite has 70 passing tests.

AccelForge Setup: Logical Cluster Memory

Architecture model

- `LogicalClusterMemory`: union of all accelerator memories.
- `NodeArray`: 2-D or 3-D accelerator grid, including TPU cube `NodeZ`.
- `HBM`: local accelerator memory.
- `UnifiedBuffer`, `Register`, `MAC`: local compute hierarchy.
- Communication scenario controls which tensors live in logical memory versus local HBM.

Workload model

Parameterized GEMM

$$\text{Outputs}_{m,n} = \sum_{k} \text{Activations}_{m,k} \, \text{Weights}_{k,n}$$

- `decode_mlp`: weight movement dominates.
- `kv_attention`: smaller attention traffic, latency-sensitive.
- `prefill_projection`: large activation and output movement.

Class connection: this is the usual accelerator hierarchy and mapper loop, except the top memory level is made network-aware instead of assumed to be a free global memory.

Network Model and Fixed-Point Feedback

Traffic cases

- `node_to_node`: activation/output exchange.
- `node_to_dram`: remote weight/state fetch.
- `shared_dram_multicast`: one logical source fans out to many nodes.

Topology cost

$$E_{\text{net}} = B_{\text{eff}} \cdot H \cdot e_{\text{bit}}, \quad L_{\text{net}} = \frac{B_{\text{eff}}}{BW/C} + HL_{\text{router}}$$

H is hop count, C is contention, and B_{eff} collapses repeated fanout only when native multicast exists.

Iterative AccelForge loop

1. Initialize logical memory as HBM-like.
2. Run AccelForge mapper.
3. Extract tensor traffic from the mapping.
4. Evaluate topology + technology network cost.
5. Add network cost to logical read/write energy and latency.
6. Rerun until costs stabilize.

Why this matters

The mapper can react to the network model. The project reports both the first-run network penalty and the final fixed-point result.

Not a pure bandwidth swap: hop count, topology contention, per-hop router latency, and native multicast all change the cost seen by the mapper.

Stalls, Overlap, and TPU-Speak

How stalls are modeled

$$T_{\text{stage}} = \max(T_{\text{compute}}, T_{\text{HBM}}) + (1 - \alpha)T_{\text{network}}$$

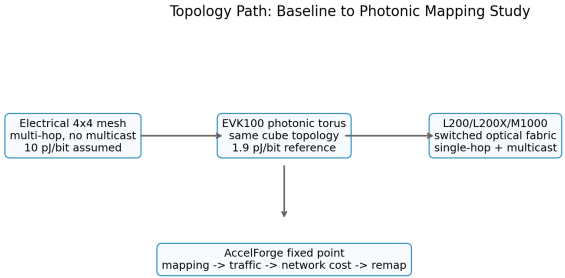
- Raw network time is only partly exposed if compute overlaps it.
- Electrical SuperPod baseline uses $\alpha = 0.35$.
- EVK100 torus + multicast reaches $\alpha = 0.58$.
- Full M1000 stack reaches $\alpha = 0.78$.

This is a rigorous analytical stall model, not a packet-level simulator with queueing, VC arbitration, or optical-switch retiming.

Translation to TPU/XLA terms

- `node_to_node`: GSPMD tensor/sequence-sharded activation exchange.
- `node_to_dram`: remote HBM, KV-cache fetch, or optimizer-state fetch.
- `shared_dram_multicast`: broadcast cache fill of shared weights or hot state.
- `remote_hbm_streaming`: pooled-HBM schedule instead of forced local residency.
- `photonic_multicast_cache`: multicast prefetch/replicated refill enabled by optics.
- `moe_locality_routing`: expert placement to reduce all-to-all pressure.

Topology Space

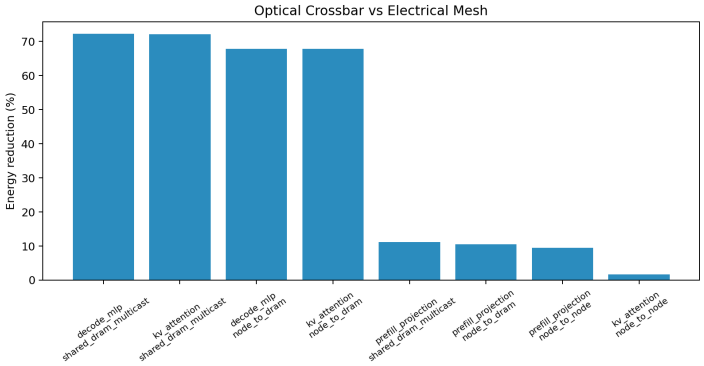


Modeled designs

- Electrical 4x4 mesh for the 16-node class-scale cluster.
- TPU-like 4x4x4 3-D torus cube.
- EVK100 photonic torus: same topology, upgraded links.
- L200/L200X/M1000 optical switched fabric: flatter single-hop fabric.
- Advanced chiplet torus: stronger electrical baseline.

The important split is topology vs link technology: EVK100 torus keeps hop structure; M1000 switched fabric changes hop structure and multicast economics.

Milestone 1 and 2: 16-Node AccelForge Study



EVK100 crossbar vs electrical mesh

Scenario	Energy	Latency
node-to-DRAM	67.91%	91.84%
node-to-node	9.40%	0.00%
shared multicast	72.21%	97.57%

Interpretation

Photonics help most when the logical memory access is remote or shared. They help least when the workload is already mostly local compute/HBM.

Milestone 2: Did the Mapping Itself Move?

Conservative AccelForge result

- In the TPU cube fixed-point mapper, photonic cases did not change compact tile/residency signatures in 24 tested cases.
- This is not a failure: the GEMM micro-mapping is dominated by local UB/register/MAC reuse for the tested tensor shapes.
- The pipeline explicitly reports this instead of hiding it.

Higher-level mapping result

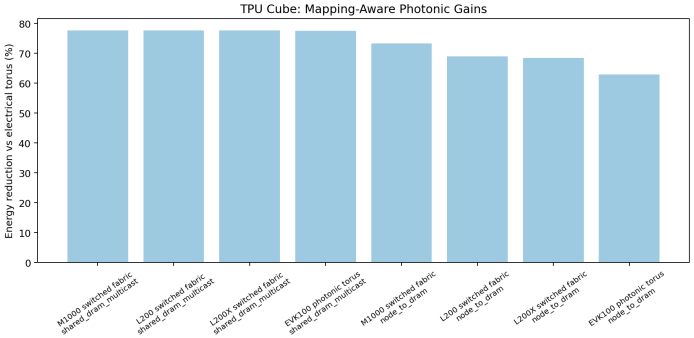
Workload	Photonic architecture	New mapping	Gain
prefill	16-node EVK100	multicast	10.08%
prefill	TPU EVK100 torus	multicast	11.36%
prefill	TPU L200 fabric	multicast	11.38%
prefill	TPU L200X fabric	multicast	11.37%
prefill	TPU M1000 fabric	multicast	11.39%

Changed distributed mappings: 5 of 16. Baseline mapping is node-to-node.

Key nuance

The inner AccelForge tile choice can remain stable while the *distributed memory placement* changes. That is exactly the compiler-level effect the project was designed to expose.

Milestone 3: TPU Cube ICI Replacement

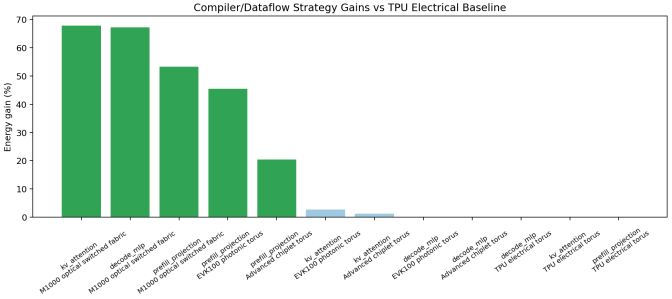


64-chip 4x4x4 cube

Variant	Best E	Best L
EVK100 torus	99.70%	99.61%
L200 switched	99.82%	99.98%
L200X switched	99.81%	99.99%
M1000 switched	99.91%	99.99%

Best cases are shared-weight/state multicast. M1000 also gives 92.45% energy and 99.65% latency reduction for all-to-all shard exchange.

Compiler/Dataflow: Where the New Paradigm Appears

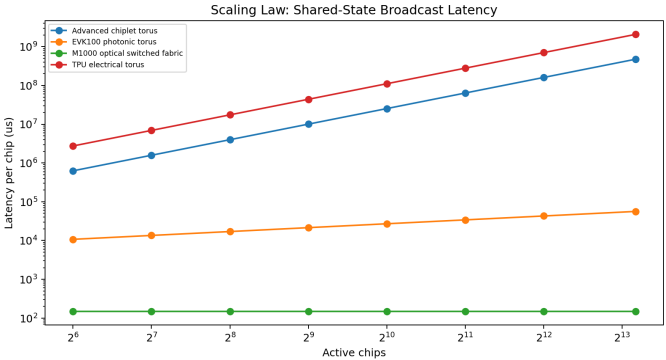


Modeled compiler choices

- XLA/GSPMD tensor-parallel collective.
- Remote pooled-HBM streaming.
- Replicated local HBM cache.
- Photonic multicast cache fill.
- MoE locality-aware routing.

Case	Best changed strategy
decode MLP / M1000	remote HBM, 67.21% E gain
KV attention / M1000	remote HBM, 67.75% E gain
prefill / EVK100	multicast cache, 45.43% E gain
prefill / M1000	remote HBM, 53.31% E gain

Scaling Laws: Why Cube Size and SuperPod Size Matter



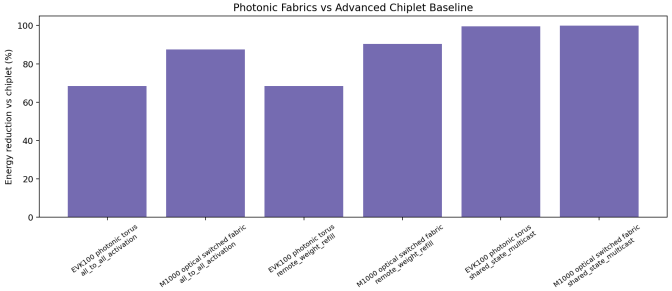
Log-log per-chip scaling exponent

Architecture / traffic	Energy	Latency
TPU torus / all-to-all	0.336	0.333
TPU torus / multicast	1.295	1.333
Chiplet torus / multicast	1.295	1.333
EVK100 torus / multicast	0.295	0.333
M1000 switched / both	0.000	0.000

Interpretation

Torus fabrics accumulate hop and contention cost as cubes grow. A flat switched optical fabric makes the modeled per-chip network cost nearly independent of system size.

Against a Stronger Baseline: Advanced Chiplet



Prompt-specified baseline

- One-way bandwidth: 3.5 Tb/s.
- Energy: 6 pJ/bit per network hop.
- No native optical multicast.

M1000 vs chiplet	Energy	Latency
all-to-all	87.42%	98.47%
weight refill	90.42%	98.47%
shared state	99.85%	99.98%

EVK100 torus is energy-efficient but can be slower than chiplet for unicast because it preserves torus hop structure.

Full 9216-Chip Study: What Is Being Claimed

Public hardware target

- TPU7x/Ironwood pod: 9216 chips.
- Per-chip public specs used in the model: 2307 BF16 TFLOP/s, 4614 FP8 TFLOP/s, 192 GiB HBM, 7380 GiB/s HBM BW, 1200 GB/s bidirectional ICI.
- Modeled baseline: Ironwood electrical torus + XLA SPMD.

Public Gemini-class proxies

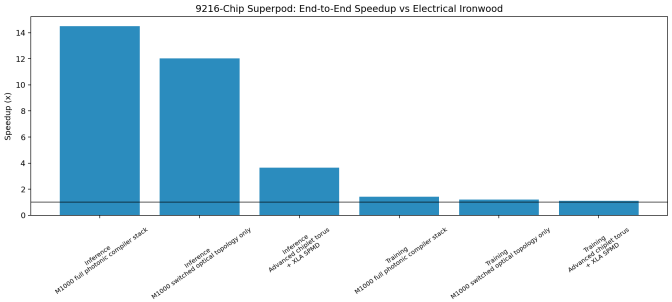
- Google does not publish Gemini 3/3.1 parameter counts or expert layouts.
- The study therefore uses explicit proxy workloads:
- sparse MoE Pro-class proxy;
- long-context Deep Think proxy;
- inference requests and training-token steps.

The deck reports modeled proxy behavior, not confidential Google internals.

Modeled configurations

Ironwood electrical torus, advanced chiplet torus, EVK100 photonic torus + multicast cache, M1000 switched topology only, and M1000 full photonic compiler stack.

End-to-End SuperPod Results



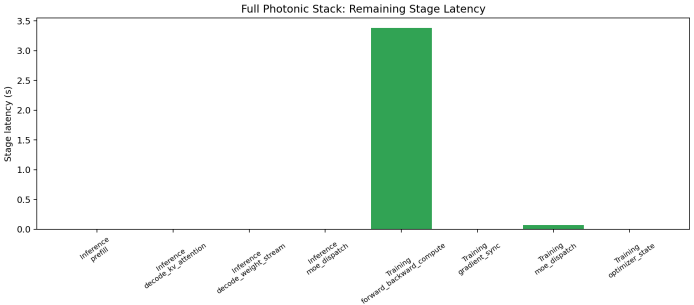
Best M1000 full-stack results

Model / phase	Speedup	Energy
Deep Think inference	14.50x	51.07%
Deep Think training	1.43x	0.67%
Gemini 3.1 inference	11.02x	45.27%
Gemini 3.1 training	1.42x	0.54%

Why inference wins harder

Inference has exposed network latency, KV/cache movement, expert routing, and multicast opportunities. Training has larger compute and HBM terms, so network removal gives smaller end-to-end energy gains.

Where the Latency Went



Network share collapse

Deep Think inference	Net E	Net L
Electrical	49.81%	90.67%
Chiplet	37.32%	67.50%
EVK100 torus+MC	9.41%	48.90%
M1000 topology only	1.23%	0.58%
M1000 full stack	0.32%	0.07%

Once network latency is mostly gone, the remaining bottleneck is compute + HBM, so further interconnect improvements have diminishing returns.

Ablation Ladder: What Each Technique Adds

Deep Think inference

Technique	Speedup	Step
baseline	1.00x	1.00x
advanced chiplet	3.65x	3.65x
photonic torus + multicast	6.09x	1.67x
switched topology only	12.02x	1.97x
multicast cache	12.76x	1.06x
remote pooled HBM	13.29x	1.04x
MoE locality routing	13.64x	1.03x
full compiler stack	14.50x	1.06x

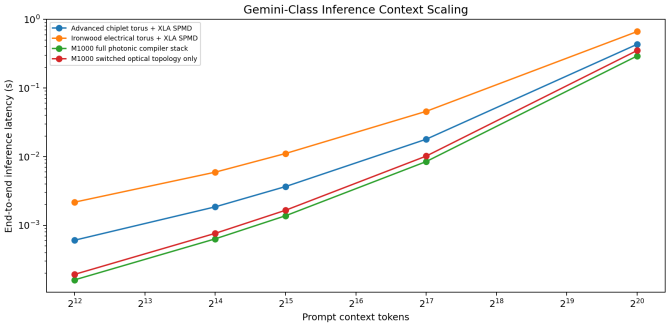
Reading the ablation

- The biggest jump is moving from electrical to high-quality chiplet or photonic links.
- The second biggest jump is topology: M1000 switched fabric eliminates most exposed network latency.
- Compiler effects are smaller in isolation, but still matter after topology because they reduce cache/expert placement overhead.
- Training has the same ordering but lower magnitude: Deep Think training reaches 1.43x.

Core conclusion

Better interconnect enables the compiler to choose remote pooled HBM, multicast cache fill, and locality routing without paying torus fanout penalties.

Long-Context Inference Scaling



Gemini 3.1 proxy context sweep

Context	M1000 speedup	Energy
4K	13.58x	58.91%
16K	9.38x	43.80%
32K	8.08x	37.58%
128K	5.40x	24.97%
1M	2.27x	6.80%

Interpretation

Photonics preserves an advantage at long context, but the model gradually becomes dominated by unavoidable compute/HBM movement rather than network fanout.

Generated Deliverables

Primary outputs

- `lightmatter_interconnects.ipynb`: executed exploratory notebook.
- `results/cluster_*.csv`: 16-node AccelForge cluster.
- `results/tpu_*.csv`: 64-chip TPU cube ICI replacement.
- `results/optimal_mapping_choices.csv`: distributed mapping selector.
- `results/compiler_strategy_study.csv`: XLA/Pallas-style strategy sweep.
- `results/scaling_*.csv`: 64-to-9216-chip scaling laws.

Presentation/report outputs

- `results/chiplet_comparison.csv`: 3.5 Tb/s / 6 pJ/bit baseline.
- `results/superpod_stage_study.csv`: stage-level model.
- `results/superpod_end_to_end_summary.csv`: total speedups.
- `results/superpod_ablation_study.csv`: technique ladder.
- `results/inference_*`, `training_*`: phase-specific workflows.
- `results/presentation/*.png`: nine condensed figures used in this deck.

The deliverables are generated, not hand-entered: re-run with `make validate`.

Limitations and Validity Checks

Validity checks

- 70 tests cover constants, topology hop counts, multicast behavior, fixed-point solver, CSV writers, notebook structure, compiler strategies, scaling, chiplet, and SuperPod workflows.
- `make validate` runs tests and regenerates the full artifact set.
- Phase-specific outputs are checked: inference-only, training-only, combined SuperPod.
- The paper source compiles after synchronization with the 70-test suite.

Modeling limits

- Not RTL, packet-level, or thermal simulation.
- Gemini workloads are explicit proxies because model internals are not public.
- Photonic multicast and OCS behavior are modeled analytically from public product specs.
- Physical integration, scheduling overheads, cost, reliability, and laser sharing are outside the current model.

Why the conclusions are still useful

The model is deliberately transparent: every assumption is in the source table or code, every result is regenerated from the repo, and the qualitative conclusion is tied to traffic shape, hop count, multicast, and compiler choices rather than a hidden black-box simulator.

Final Takeaways

Not universal

Training speedup is modest

Best modeled training speedup is 1.43x because compute dominates after collectives are reduced.

Very strong for inference

Network-heavy stages move

Best modeled inference speedup is 14.50x; network latency share falls from 90.67% to 0.07%.

New mappings

The compiler should adapt

Shared multicast, pooled HBM, cache fill, and MoE locality become attractive under photonics.

Bottom line

Lightmatter-style photonic interconnects are most compelling when they change the memory/dataflow problem: large fanout, long context, expert routing, shared state, and remote cache movement. The project shows both the constant-factor link gains and the deeper mapping/compiler gains needed for a convincing accelerator-architecture story.

Backup: Figure Index

Figure	Meaning
`01_cluster_reductions.png`	16-node EVK100 optical crossbar gains.
`02_tpu_mapping_reductions.png`	TPU cube fixed-point mapping-aware gains.
`03_topology_diagram.png`	Topology comparison.
`04_compiler_strategy_choices.png`	XLA/Pallas-style strategy changes.
`05_scaling_laws.png`	64-to-9216-chip scaling-law slopes.
`06_chiplet_baseline.png`	Advanced chiplet comparison.
`07_superpod_end_to_end_speedups.png`	Full-SuperPod speedups.
`08_superpod_stage_breakdown.png`	SuperPod latency/energy breakdown.
`09_superpod_context_scaling.png`	Long-context inference scaling.

Backup: Reproduction Checklist

Environment

- Repo root: `final-project/`
- Docker image: `timeloopaccelergy/accelforge:latest-amd64`
- Main working directory inside container: `/home/workspace/workspace/final_project`

Commands

- `make test`: 70-test validation suite.
- `make study`: all CSV, Markdown, and plot artifacts.
- `make inference/training/superpod`: phase workflows.
- `make notebook`: regenerate and execute notebook.
- `make validate`: production path used for this deck.